Sistemes Operatius

Tema 3: Concurrència, exclusió mútua i mutex · Curs 2026-27

LA SALLE
UNIVERSITAT RAMON LLULL
ESCOLA TÈCNICA SUPERIOR D'ENGINYERIA

Dos models per intercanviar dades

Memòria compartida

Els processos accedeixen al mateix estat. És eficient, però el programa ha de protegir els recursos compartits i coordinar-ne els accessos.

Pas de missatges

Cada procés manté memòria privada i intercanvia còpies amb send/receive. El sistema operatiu gestiona el canal i els possibles bloqueigs.

El problema general de la concurrència

Execucions concurrents

Dos o més fils o processos progressen durant el mateix interval. En un sol nucli s'intercalen; en diversos nuclis també poden executar-se en paral·lel.

Recurs compartit

És qualsevol dada, dispositiu o estructura accessible des de més d'una execució: una variable, un buffer, un fitxer o un perifèric.

Intercalat no determinista

El planificador pot aturar una execució entre dues instruccions i reprendre'n una altra. El programa no pot dependre d'un ordre favorable.

No cal simultaneïtat exacta: n'hi ha prou que operacions sobre el mateix recurs es puguin intercalar.

Secció crítica: mantenir consistent el recurs compartit

Una secció crítica és el fragment que accedeix a un recurs compartit i que no pot solapar-se amb un accés incompatible.

Entrada

Adquirir el mecanisme abans de tocar l'estat.

Secció crítica

Llegir, comprovar i actualitzar mantenint consistent el recurs compartit.

Sortida

Alliberar sempre, també en camins d'error.

Exclusió mútua: la propietat que necessitem

Objectiu de seguretat

Com a màxim una execució incompatible pot estar dins la secció crítica associada al recurs. Així s'eviten actualitzacions parcials o perdudes.

Un protocol compartit

Tots els accessos incompatibles al mateix recurs han d'utilitzar el mateix mecanisme d'entrada i sortida. Protegir només alguns camins no és correcte.

Què no garanteix

Evita el solapament, però per si sola no assegura progrés, espera limitada, equitat ni absència d'interbloqueig.

Exclusió mútua és una propietat de la solució; un mutex és una eina concreta per implementar-la.

El problema: dos fils, una variable compartida

Dos fils, el mateix codi

```
long glob = 0;

void *compta(void *arg) {
  for (int i = 0; i < N; i++)
    glob++;
}
```

Què esperaríem?

- **Dos fils:** tots dos executen compta() sobre la mateixa variable global.
- **Resultat esperat:** glob == 2 × N.
- **Resultat observat:** algunes execucions donen un valor inferior.
- **Pregunta:** com es pot perdre un increment si cada fil executa exactament N iteracions?

La creació dels fils és correcta; el problema és l'actualització no protegida de l'estat compartit.

Compartir estat: el límit de l'atomicitat

```
glob++  equival aproximadament a:

tmp = llegeix(glob)
tmp = tmp + 1
escriu(glob, tmp)
```

- **Atomicitat:** una operació es veu com un únic pas indivisible per la resta d'execucions.
- **Read–modify–write:** una expressió de C pot contenir diversos accessos i no és atòmica per defecte.
- **Conflicte:** dos accessos són incompatibles si toquen la mateixa dada i almenys un l'escriu.

Compartir una variable no és el problema per si sol; el risc apareix quan accessos incompatibles poden solapar-se.

Condició de carrera: actualització perduda

```
Valor inicial: glob = 7

Fil A: llegeix 7 → calcula 8
Fil B: llegeix 7 → calcula 8
Fil A: escriu 8
Fil B: escriu 8
```

- **Resultat observat:** glob val 8 encara que s'han executat dos increments.
- **Causa:** ambdós fils han calculat a partir del mateix valor antic.
- **Definició:** el resultat correcte depèn de l'ordre temporal dels accessos concurrents.

La carrera pot desaparèixer en una prova i tornar a aparèixer en una altra

Com detectar una condició de carrera

1. Mateixa dada

Localitza l'estat accessible des de més d'una execució.

2. Solapament possible

Comprova si els accessos poden executar-se sense un ordre imposat.

3. Almenys una escriptura

Lectura–lectura és compatible, lectura–escriptura i escriptura–escriptura no ho són.

Exclusió mútua i sincronització no són el mateix

Exclusió mútua

Impedeix que accessos incompatibles entrin simultàniament en una secció crítica. És una propietat de seguretat.

Sincronització

Imposa una dependència: una execució espera fins que una altra produeix un esdeveniment o un recurs.

Què exigeix una solució correcta?

Exclusió mútua (seguretat)

Mai no hi ha dues execucions incompatibles dins la mateixa secció crítica.

Progrés

Si la secció és lliure i hi ha candidats, algun d'ells ha de poder avançar.

Espera limitada

Cap candidat pot ser ajornat indefinidament mentre la resta hi entra repetidament.

Sense supòsits temporals

La correcció no depèn de velocitats relatives, del nombre de nuclis ni d'un ordre favorable.

Les operacions d'entrada i sortida del mecanisme han de ser atòmiques.

Què és exactament un mutex?

Objecte amb estat

Un mutex representa un únic propietari temporal. Pot estar lliure o adquirit i s'associa conceptualment al recurs compartit que protegeix.

lock

Si és lliure, el fil l'adquireix i entra. Si està ocupat, lock espera fins que pugui adquirir-lo; retornar de lock concedeix el permís d'entrada.

unlock i propietat

Només el fil que ha adquirit el mutex l'ha d'alliberar. unlock s'ha d'executar en tots els camins, també quan hi ha errors.

Mutex vol dir mutual exclusion: protegeix un recurs compartit seguint el protocol lock → secció crítica → unlock.

De l'atomicitat al mutex

```
mutex m
init(m)

lock(m)
/* secció crítica */
unlock(m)
```

- **Base hardware:** el lock descansa en una operació read–modify–write atòmica, com test-and-set o compare-and-swap.
- **Per què no la programem nosaltres:** desactivar interrupcions és privilegiat i insuficient en multiprocessador; Peterson o Dekker són models històrics amb espera activa i supòsits estrictes.
- **Abstracció útil:** el mutex combina estat atòmic amb bloqueig i despertar gestionats per la llibreria o el sistema.
- **Protocol:** tots els accessos incompatibles usen el mateix mutex; qui fa lock és qui fa unlock.

Mutex amb threads: l'eina en C

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

void *compta(void *arg) {
  for (int i = 0; i < N; i++) {
    pthread_mutex_lock(&m);
    glob++;
    pthread_mutex_unlock(&m);
  }
  return NULL;
}

/* després dels pthread_join */
pthread_mutex_destroy(&m);
```

- **Un mutex per recurs compartit: tots els fils protegeixen glob amb el mateix objecte.**
- **Propietari: el fil que completa lock és qui executa unlock.**
- **Sense espera activa al codi: lock pot bloquejar el fil fins que el mutex quedi lliure.**
- **Cicle de vida: inicialitzar abans de crear fils i destruir després dels join.**

Mutex per protegir memòria entre fils; semàfor quan cal comptar permisos, ordenar esdeveniments o coordinar processos.

Mutex en C: cicle de vida i errors

Crear els fils

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_t t1, t2;
int s;

s = pthread_create(&t1, NULL, compta, &loops);

s = pthread_create(&t2, NULL, compta, &loops);
```

Esperar i destruir

```
s = pthread_join(t1, NULL);

s = pthread_join(t2, NULL);

s = pthread_mutex_destroy(&m);
```

Interbloqueig, inanició i equitat

Interbloqueig (deadlock)

A té R1 i espera R2; B té R2 i espera R1. Cap pot continuar. Un ordre global de recursos pot prevenir-ho.

Inanició (starvation)

Una execució preparada queda ajornada indefinidament perquè altres obtenen repetidament el recurs.

Equitat (fairness)

Una política evita ajornaments indefinits. No implica necessàriament FIFO.

Exclusió mútua evita solapaments; no garanteix per si sola progrés ni equitat.